

**BABU BANARASI DAS**  
**UNIVERSITY**

**2026-2027**



**SCHOOL OF COMPUTER APPLICATIONS**

**NO SQL PROJECT ON**  
**Student Management System Using MongoDB**

**Submitted to:**

Mr. Harendra Singh

**Department:** BCA (DS & AI)

**Submitted by:**

Jagat Singh

**Roll No:** 62

**University Roll No:** 1250258229

**Section:** BCADS23

## **ACKNOWLEDGEMENT**

I sincerely express my gratitude to my teacher Mr. Harendra Singh for giving me the opportunity to work on this NoSQL project titled “Student Management System Using MongoDB”.

This project helped me understand the basic concepts of NoSQL databases and learn how MongoDB is used to create, retrieve, update, sort, and delete data. I also learned how different query operators such as greater than, less than, equal to, AND, and OR are used in MongoDB.

I am thankful to my teacher for the guidance and support provided throughout the completion of this project.

## **INTRODUCTION**

MongoDB is a popular NoSQL database management system that stores data in the form of documents. Unlike traditional relational databases, MongoDB does not require data to be stored in fixed rows and columns.

In this project, a Student Management System has been created using MongoDB. The database contains information about 30 students, including their roll number, name, age, marks, and city.

Different MongoDB operations have been performed on the student data, such as inserting, displaying, updating, deleting, sorting, counting, and searching records. Comparison operators such as \$eq, \$gt, \$lt, \$gte, and \$lte have also been demonstrated.

## **OBJECTIVES**

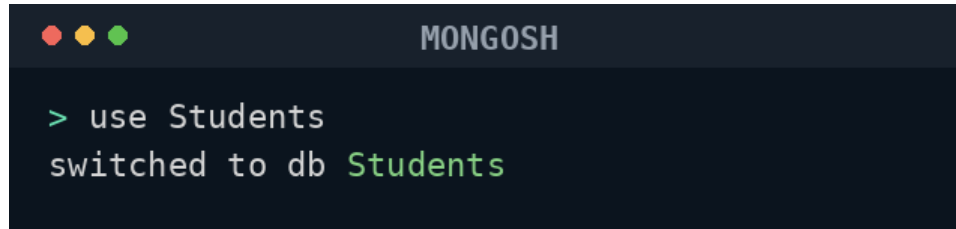
- To understand the concept of NoSQL databases.
- To learn the basic features of MongoDB.
- To create a database and collection.
- To insert 30 student records.
- To perform CRUD operations.
- To use comparison operators.
- To perform sorting and filtering.
- To understand AND and OR operations.

## 1. CREATE DATABASE

**MongoDB Query:**

*use Students*

**Explanation:** Creates or switches to the Student database.



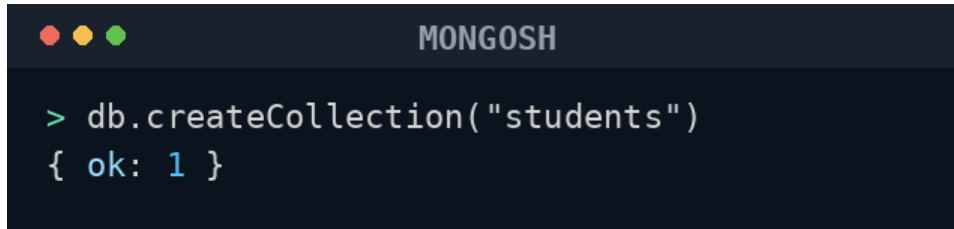
```
MONGOSH  
  
> use Students  
switched to db Students
```

## 2. CREATE COLLECTION

**MongoDB Query:**

```
db.createCollection("students")
```

**Explanation:** Creates a collection named students.

A terminal window titled 'MONGOSH' with three colored window control buttons (red, yellow, green) in the top-left corner. The terminal displays a MongoDB command and its output.

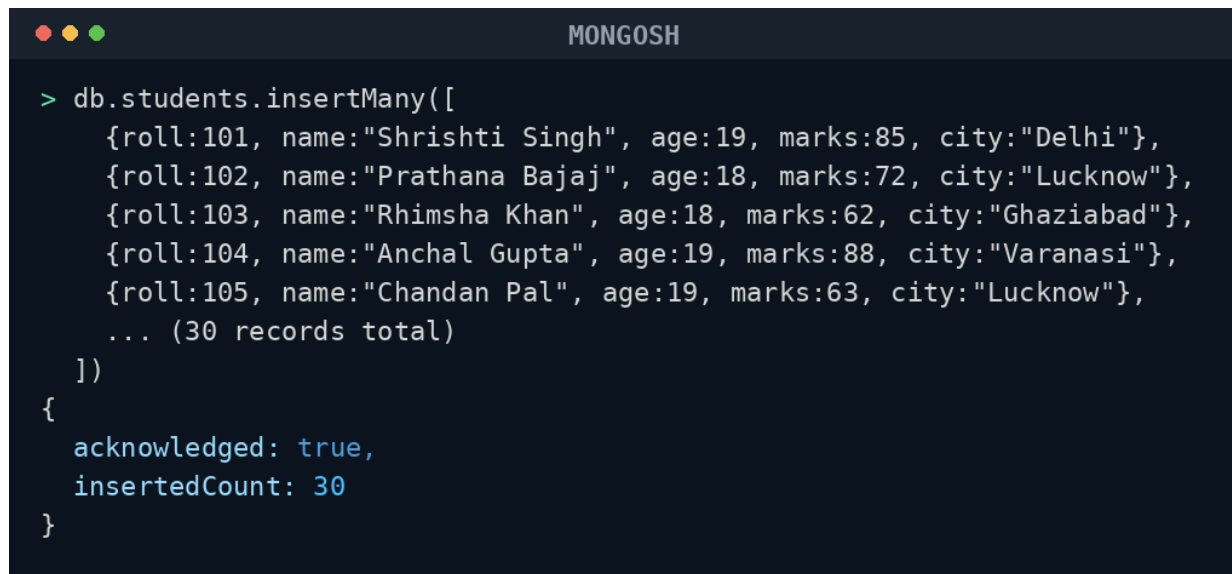
```
> db.createCollection("students")
{ ok: 1 }
```

### 3. INSERT 30 STUDENTS

**MongoDB Query:**

```
db.students.insertMany([
  {roll:101, name:"Shrishti Singh", age:19, marks:85, city:"Delhi"},
  {roll:102, name:"Prathana Bajaj", age:18, marks:72, city:"Lucknow"},
  {roll:103, name:"Rhimsha Khan", age:18, marks:62, city:"Ghaziabad"},
  {roll:104, name:"Anchal Gupta", age:19, marks:88, city:"Varanasi"},
  {roll:105, name:"Chandan Pal", age:19, marks:63, city:"Lucknow"},
  {roll:106, name:"Ishika Shrivastava", age:19, marks:83, city:"Prayagraj"},
  {roll:107, name:"Kritika Verma", age:18, marks:65, city:"Ghaziabad"},
  {roll:108, name:"Saksham Rao", age:19, marks:71, city:"Ghaziabad"},
  {roll:109, name:"Devyani Joshi", age:19, marks:93, city:"Ghaziabad"},
  {roll:110, name:"Aryan Malhotra", age:19, marks:61, city:"Varanasi"},
  {roll:111, name:"Nikhil Ali", age:20, marks:54, city:"Ghaziabad"},
  {roll:112, name:"Rohan Tripathi", age:20, marks:95, city:"Prayagraj"},
  {roll:113, name:"Tanisha Kapoor", age:19, marks:57, city:"Delhi"},
  {roll:114, name:"Faizan Ahmed", age:19, marks:85, city:"Prayagraj"},
  {roll:115, name:"Vanshika Rawat", age:20, marks:60, city:"Lucknow"},
  {roll:116, name:"Lakshay Sethi", age:18, marks:61, city:"Noida"},
  {roll:117, name:"Muskan Jaiswal", age:19, marks:84, city:"Varanasi"},
  {roll:118, name:"Harshvardhan Yadav", age:20, marks:90, city:"Ghaziabad"},
  {roll:119, name:"Gungun Mishra", age:20, marks:94, city:"Lucknow"},
  {roll:120, name:"Sarthak Bhatt", age:20, marks:61, city:"Prayagraj"},
  {roll:121, name:"Alisha Khan", age:20, marks:95, city:"Noida"},
  {roll:122, name:"Divyanshu Pandey", age:19, marks:93, city:"Lucknow"},
  {roll:123, name:"Kashish Arora", age:20, marks:94, city:"Delhi"},
  {roll:124, name:"Rajat Chauhan", age:19, marks:81, city:"Delhi"},
  {roll:125, name:"Shivangi Dubey", age:20, marks:61, city:"Ghaziabad"},
  {roll:126, name:"Yashwant Rana", age:18, marks:60, city:"Delhi"},
  {roll:127, name:"Bhavya Saxena", age:19, marks:73, city:"Ghaziabad"},
  {roll:128, name:"Nitya Agarwal", age:18, marks:63, city:"Delhi"},
  {roll:129, name:"Om Prakash", age:19, marks:95, city:"Ghaziabad"},
  {roll:130, name:"Tanmay Srivastava", age:19, marks:72, city:"Varanasi"}
])
```

**Explanation:** Inserts multiple student documents into the collection.

A terminal window with a dark background and light-colored text. The title bar at the top shows three colored circles (red, yellow, green) on the left and the word "MONGOSH" in the center. The terminal displays a MongoDB command to insert 30 records into the 'students' collection. The command is: > db.students.insertMany([ {roll:101, name:"Shrishti Singh", age:19, marks:85, city:"Delhi"}, {roll:102, name:"Prathana Bajaj", age:18, marks:72, city:"Lucknow"}, {roll:103, name:"Rhimsha Khan", age:18, marks:62, city:"Ghaziabad"}, {roll:104, name:"Anchal Gupta", age:19, marks:88, city:"Varanasi"}, {roll:105, name:"Chandan Pal", age:19, marks:63, city:"Lucknow"}, ... (30 records total) ]). The output shows a JSON object with "acknowledged: true" and "insertedCount: 30".

```
> db.students.insertMany([
  {roll:101, name:"Shrishti Singh", age:19, marks:85, city:"Delhi"},
  {roll:102, name:"Prathana Bajaj", age:18, marks:72, city:"Lucknow"},
  {roll:103, name:"Rhimsha Khan", age:18, marks:62, city:"Ghaziabad"},
  {roll:104, name:"Anchal Gupta", age:19, marks:88, city:"Varanasi"},
  {roll:105, name:"Chandan Pal", age:19, marks:63, city:"Lucknow"},
  ... (30 records total)
])
{
  acknowledged: true,
  insertedCount: 30
}
```

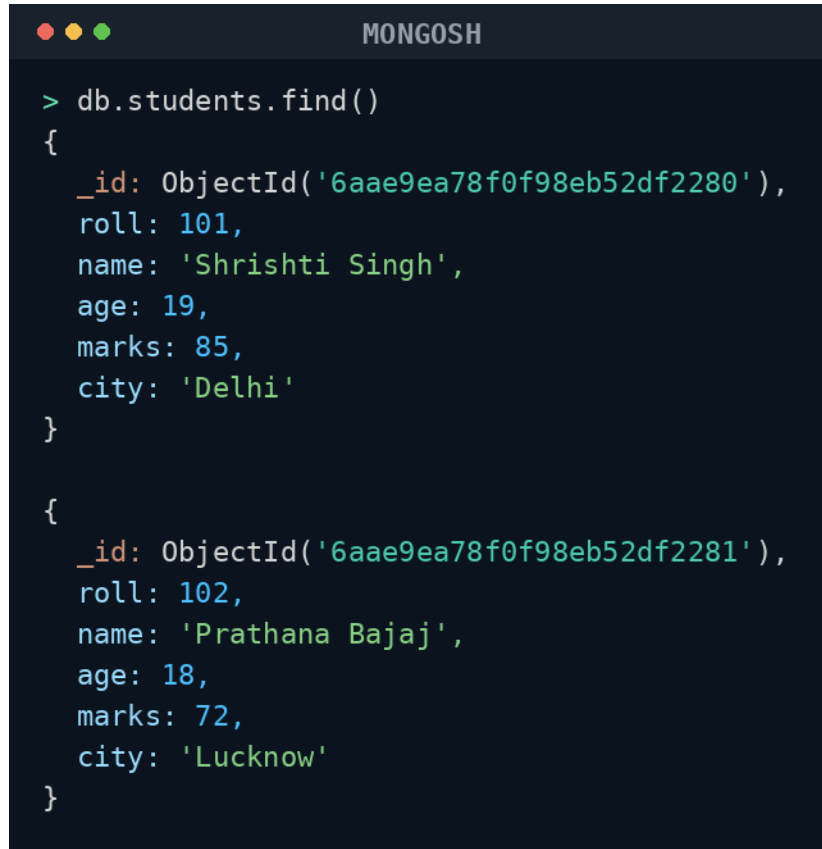
*All 30 student records were inserted successfully into the 'students' collection.*

## 4. DISPLAY ALL STUDENTS

**MongoDB Query:**

`db.students.find()`

**Explanation:** Retrieves documents from the students collection.

A screenshot of a terminal window titled "MONGOSH". The terminal shows the command `> db.students.find()` and its output, which consists of two JSON documents. The first document represents a student with roll number 101, name 'Shrishti Singh', age 19, marks 85, and city 'Delhi'. The second document represents a student with roll number 102, name 'Prathana Bajaj', age 18, marks 72, and city 'Lucknow'.

```
> db.students.find()
{
  _id: ObjectId('6aae9ea78f0f98eb52df2280'),
  roll: 101,
  name: 'Shrishti Singh',
  age: 19,
  marks: 85,
  city: 'Delhi'
}

{
  _id: ObjectId('6aae9ea78f0f98eb52df2281'),
  roll: 102,
  name: 'Prathana Bajaj',
  age: 18,
  marks: 72,
  city: 'Lucknow'
}
```

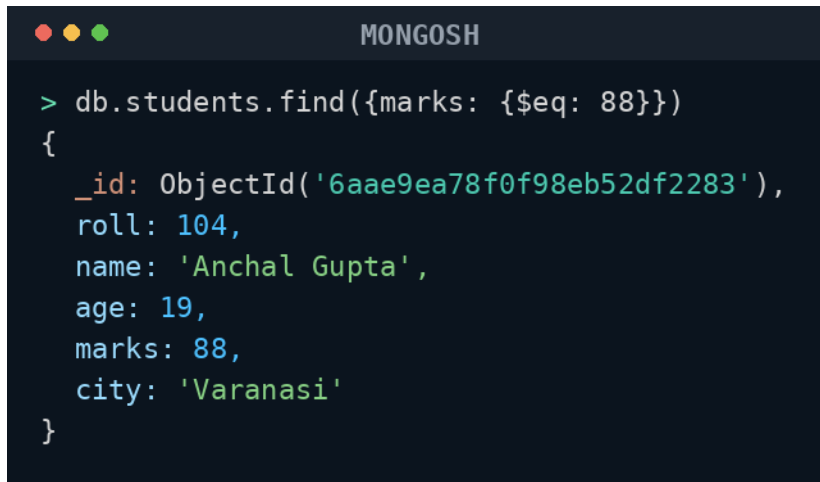
*Showing the first 2 of 30 documents returned by this query.*

## 5. EQUAL TO — \$eq

**MongoDB Query:**

```
db.students.find({marks: {$eq: 88}})
```

**Explanation:** Finds students whose marks are exactly 88.



```
MONGOSH
> db.students.find({marks: {$eq: 88}})
{
  _id: ObjectId('6aae9ea78f0f98eb52df2283'),
  roll: 104,
  name: 'Anchal Gupta',
  age: 19,
  marks: 88,
  city: 'Varanasi'
}
```

## 6. GREATER THAN — \$gt

MongoDB Query:

```
db.students.find({marks: {$gt: 85}})
```

**Explanation:** Finds students whose marks are greater than 85.

```
MONGOSH

> db.students.find({marks: {$gt: 85}})
{
  _id: ObjectId('6aae9ea78f0f98eb52df2283'),
  roll: 104,
  name: 'Anchal Gupta',
  age: 19,
  marks: 88,
  city: 'Varanasi'
}

{
  _id: ObjectId('6aae9ea78f0f98eb52df2288'),
  roll: 109,
  name: 'Devyani Joshi',
  age: 19,
  marks: 93,
  city: 'Ghaziabad'
}

{
  _id: ObjectId('6aae9ea78f0f98eb52df228b'),
  roll: 112,
  name: 'Rohan Tripathi',
  age: 20,
  marks: 95,
  city: 'Prayagraj'
}
```

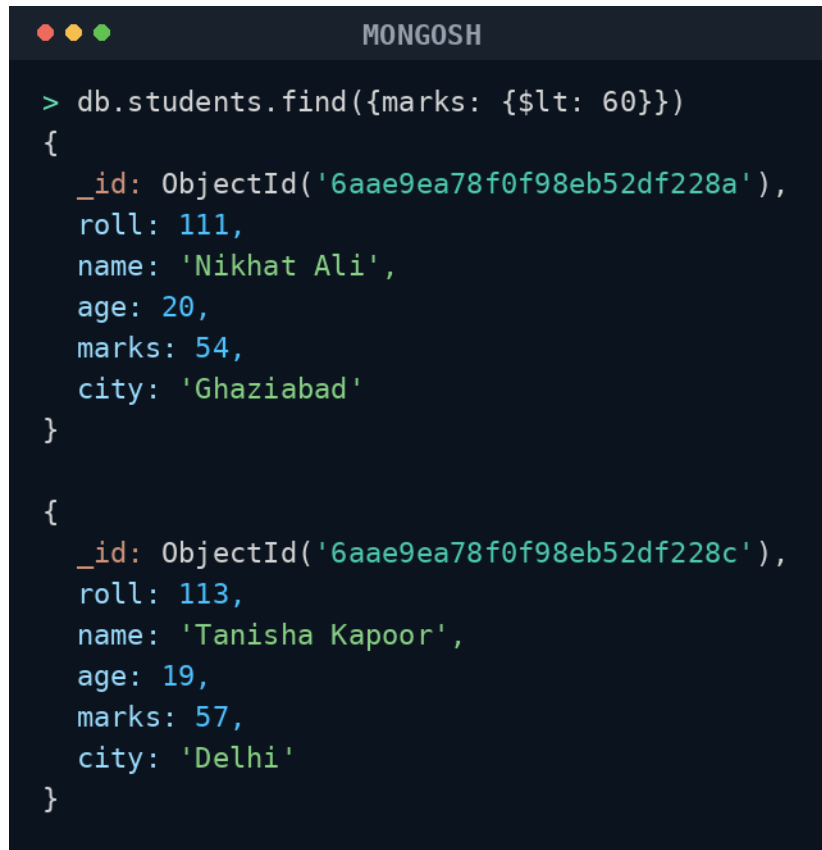
Showing 3 of 9 matching documents.

## 7. LESS THAN — \$lt

**MongoDB Query:**

`db.students.find({marks: {$lt: 60}})`

**Explanation:** Finds students whose marks are less than 60.



```
MONGOSH
> db.students.find({marks: {$lt: 60}})
{
  _id: ObjectId('6aae9ea78f0f98eb52df228a'),
  roll: 111,
  name: 'Nikhath Ali',
  age: 20,
  marks: 54,
  city: 'Ghaziabad'
}

{
  _id: ObjectId('6aae9ea78f0f98eb52df228c'),
  roll: 113,
  name: 'Tanisha Kapoor',
  age: 19,
  marks: 57,
  city: 'Delhi'
}
```

## 8. GREATER THAN OR EQUAL TO — \$gte

MongoDB Query:

```
db.students.find({marks: {$gte: 90}})
```

**Explanation:** Finds students whose marks are 90 or above.

```
MONGOSH

> db.students.find({marks: {$gte: 90}})
{
  _id: ObjectId('6aae9ea78f0f98eb52df2288'),
  roll: 109,
  name: 'Devyani Joshi',
  age: 19,
  marks: 93,
  city: 'Ghaziabad'
}

{
  _id: ObjectId('6aae9ea78f0f98eb52df228b'),
  roll: 112,
  name: 'Rohan Tripathi',
  age: 20,
  marks: 95,
  city: 'Prayagraj'
}

{
  _id: ObjectId('6aae9ea78f0f98eb52df2291'),
  roll: 118,
  name: 'Harshvardhan Yadav',
  age: 20,
  marks: 90,
  city: 'Ghaziabad'
}
```

Showing 3 of 8 matching documents.

## 9. LESS THAN OR EQUAL TO — \$lte

**MongoDB Query:**

```
db.students.find({marks: {$lte: 70}})
```

**Explanation:** Finds students whose marks are 70 or below.

```
MONGOSH

> db.students.find({marks: {$lte: 70}})
{
  _id: ObjectId('6aae9ea78f0f98eb52df2282'),
  roll: 103,
  name: 'Rhimsha Khan',
  age: 18,
  marks: 62,
  city: 'Ghaziabad'
}

{
  _id: ObjectId('6aae9ea78f0f98eb52df2284'),
  roll: 105,
  name: 'Chandan Pal',
  age: 19,
  marks: 63,
  city: 'Lucknow'
}

{
  _id: ObjectId('6aae9ea78f0f98eb52df2286'),
  roll: 107,
  name: 'Kritika Verma',
  age: 18,
  marks: 65,
  city: 'Ghaziabad'
}
```

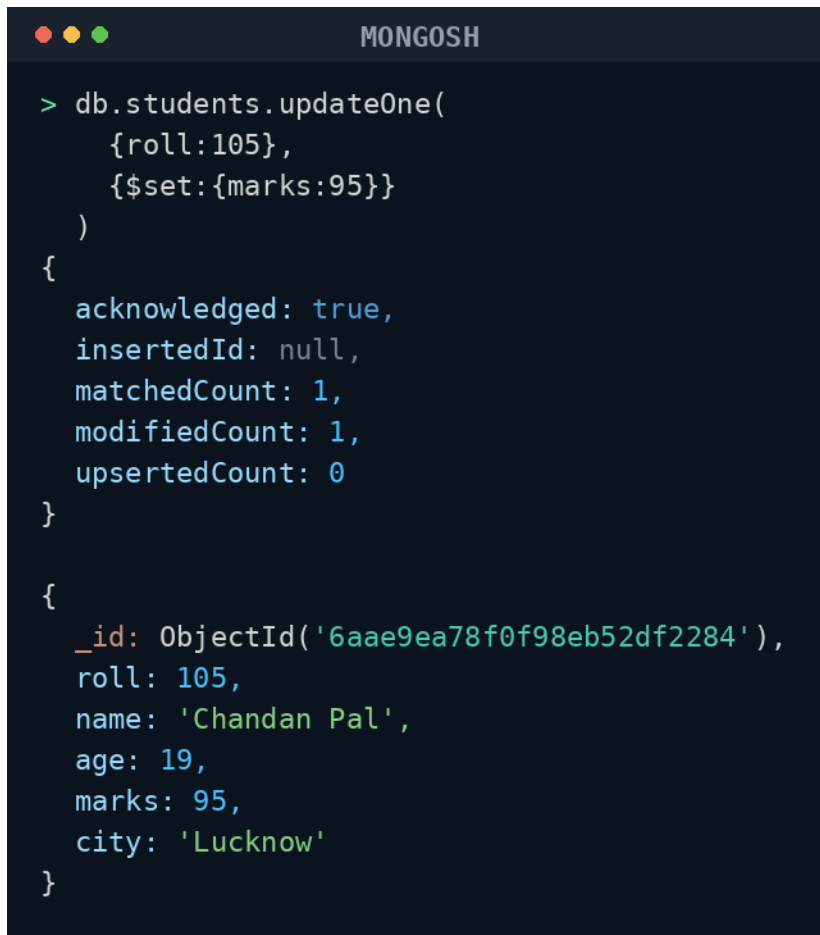
*Showing 3 of 12 matching documents.*

## 10. UPDATE OPERATION

**MongoDB Query:**

```
db.students.updateOne(  
  {roll:105},  
  {$set:{marks:95}}  
)
```

**Explanation:** Updates the marks to 95 of the student with roll number 105.



```
MONGOSH  
  
> db.students.updateOne(  
  {roll:105},  
  {$set:{marks:95}}  
)  
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}  
  
{  
  _id: ObjectId('6aae9ea78f0f98eb52df2284'),  
  roll: 105,  
  name: 'Chandan Pal',  
  age: 19,  
  marks: 95,  
  city: 'Lucknow'  
}
```

## 11. DELETE ONE OPERATION

**MongoDB Query:**

*db.students.deleteOne({roll:110})*

**Explanation:** Deletes one document matching roll number 110.

A terminal window titled 'MONGOSH' with three colored window control buttons (red, yellow, green) in the top-left corner. The terminal displays a MongoDB command and its output. The command is entered on the first line, and the output is displayed on the subsequent lines.

```
> db.students.deleteOne({roll:110})
{
  acknowledged: true,
  deletedCount: 1
}
```

## 12. DELETE MANY OPERATION

**MongoDB Query:**

```
db.students.deleteMany({  
  marks: {$lt:60}  
})
```

**Explanation:** Deletes all students having marks below 60.

A screenshot of a terminal window titled "MONGOSH". The terminal shows a MongoDB query being executed: 

```
> db.students.deleteMany({  
  marks: {$lt:60}  
})  
{  
  acknowledged: true,  
  deletedCount: 2  
}
```

 The output indicates that 2 documents were successfully deleted.

*Removed roll numbers 111, 113 (marks below 60).*

### 13. AND OPERATION — \$and

MongoDB Query:

```
db.students.find({
  $and:[
    {marks:{$gt:80}},
    {age:19}
  ]
})
```

**Explanation:** Finds students satisfying both conditions.

A screenshot of a terminal window titled "MONGOSH". The terminal shows a MongoDB query being executed: `> db.students.find({ $and:[ {marks:{$gt:80}}, {age:19} ] })`. The output displays two JSON documents. The first document has fields: `_id` (ObjectId), `roll` (101), `name` ('Shrishti Singh'), `age` (19), `marks` (85), and `city` ('Delhi'). The second document has fields: `_id` (ObjectId), `roll` (104), `name` ('Anchal Gupta'), `age` (19), `marks` (88), and `city` ('Varanasi').

```
MONGOSH
> db.students.find({
  $and:[
    {marks:{$gt:80}},
    {age:19}
  ]
})
{
  _id: ObjectId('6aae9ea78f0f98eb52df2280'),
  roll: 101,
  name: 'Shrishti Singh',
  age: 19,
  marks: 85,
  city: 'Delhi'
}

{
  _id: ObjectId('6aae9ea78f0f98eb52df2283'),
  roll: 104,
  name: 'Anchal Gupta',
  age: 19,
  marks: 88,
  city: 'Varanasi'
}
```

Showing 2 of 10 matching documents.

## 14. OR OPERATION — \$or

MongoDB Query:

```
db.students.find({
  $or:[
    {city:"Lucknow"},
    {city:"Delhi"}
  ]
})
```

**Explanation:** Finds students from Lucknow or Delhi.

A screenshot of a terminal window titled "MONGOSH". The terminal shows a MongoDB query being executed: `> db.students.find({ $or:[ {city:"Lucknow"}, {city:"Delhi"} ] })`. The output displays two JSON documents. The first document is for a student with `_id: ObjectId('6aae9ea78f0f98eb52df2280')`, `roll: 101`, `name: 'Shrishti Singh'`, `age: 19`, `marks: 85`, and `city: 'Delhi'`. The second document is for a student with `_id: ObjectId('6aae9ea78f0f98eb52df2281')`, `roll: 102`, `name: 'Prathana Bajaj'`, `age: 18`, `marks: 72`, and `city: 'Lucknow'`.

```
> db.students.find({
  $or:[
    {city:"Lucknow"},
    {city:"Delhi"}
  ]
})
{
  _id: ObjectId('6aae9ea78f0f98eb52df2280'),
  roll: 101,
  name: 'Shrishti Singh',
  age: 19,
  marks: 85,
  city: 'Delhi'
}

{
  _id: ObjectId('6aae9ea78f0f98eb52df2281'),
  roll: 102,
  name: 'Prathana Bajaj',
  age: 18,
  marks: 72,
  city: 'Lucknow'
}
```

Showing 2 of 10 matching documents.

## 15. SORT OPERATION

### A. MongoDB Query:

```
db.students.find().sort({marks : 1})
```

**Explanation:** Sorts students by marks from lowest to highest. Use 1 for ascending order.

A screenshot of a terminal window titled "MONGOSH". The terminal shows a MongoDB query being executed: `> db.students.find().sort({marks : 1})`. The output displays three student records in JSON format, sorted by marks in ascending order. The first record is for Vanshika Rawat (roll: 115, marks: 60, city: Lucknow). The second record is for Yashwant Rana (roll: 126, marks: 60, city: Delhi). The third record is for Lakshay Sethi (roll: 116, marks: 61, city: Noida).

```
> db.students.find().sort({marks : 1})
{
  _id: ObjectId('6aae9ea78f0f98eb52df228e'),
  roll: 115,
  name: 'Vanshika Rawat',
  age: 20,
  marks: 60,
  city: 'Lucknow'
}

{
  _id: ObjectId('6aae9ea78f0f98eb52df2299'),
  roll: 126,
  name: 'Yashwant Rana',
  age: 18,
  marks: 60,
  city: 'Delhi'
}

{
  _id: ObjectId('6aae9ea78f0f98eb52df228f'),
  roll: 116,
  name: 'Lakshay Sethi',
  age: 18,
  marks: 61,
  city: 'Noida'
}
```

### B. MongoDB Query:

```
db.students.find().sort({marks: -1})
```

**Explanation:** Sorts students by marks from highest to lowest. Use -1 for descending order.

## MONGOSH

```
> db.students.find().sort({marks: -1})
{
  _id: ObjectId('6aae9ea78f0f98eb52df2284'),
  roll: 105,
  name: 'Chandan Pal',
  age: 19,
  marks: 95,
  city: 'Lucknow'
}

{
  _id: ObjectId('6aae9ea78f0f98eb52df228b'),
  roll: 112,
  name: 'Rohan Tripathi',
  age: 20,
  marks: 95,
  city: 'Prayagraj'
}

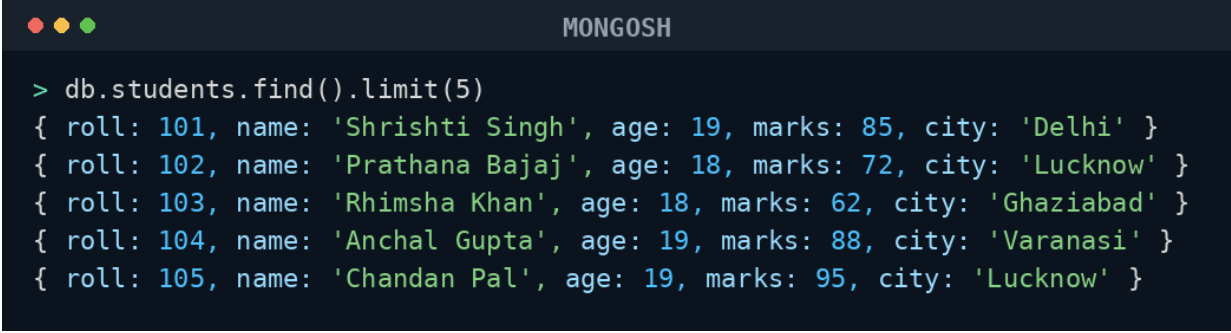
{
  _id: ObjectId('6aae9ea78f0f98eb52df2294'),
  roll: 121,
  name: 'Alisha Khan',
  age: 20,
  marks: 95,
  city: 'Noida'
}
```

## 16. LIMIT OPERATION

**MongoDB Query:**

`db.students.find().limit(5)`

**Explanation:** Displays only the first five documents.



```
> db.students.find().limit(5)
{ roll: 101, name: 'Shrishti Singh', age: 19, marks: 85, city: 'Delhi' }
{ roll: 102, name: 'Prathana Bajaj', age: 18, marks: 72, city: 'Lucknow' }
{ roll: 103, name: 'Rhimsha Khan', age: 18, marks: 62, city: 'Ghaziabad' }
{ roll: 104, name: 'Anchal Gupta', age: 19, marks: 88, city: 'Varanasi' }
{ roll: 105, name: 'Chandan Pal', age: 19, marks: 95, city: 'Lucknow' }
```

## 17. SEARCH BY CITY

**MongoDB Query:**

```
db.students.find({city:"Lucknow"})
```

**Explanation:** Finds all students whose city is Lucknow.



```
> db.students.find({city:"Lucknow"})
{
  _id: ObjectId('6aae9ea78f0f98eb52df2281'),
  roll: 102,
  name: 'Prathana Bajaj',
  age: 18,
  marks: 72,
  city: 'Lucknow'
}

{
  _id: ObjectId('6aae9ea78f0f98eb52df2284'),
  roll: 105,
  name: 'Chandan Pal',
  age: 19,
  marks: 95,
  city: 'Lucknow'
}

{
  _id: ObjectId('6aae9ea78f0f98eb52df228e'),
  roll: 115,
  name: 'Vanshika Rawat',
  age: 20,
  marks: 60,
  city: 'Lucknow'
}
```

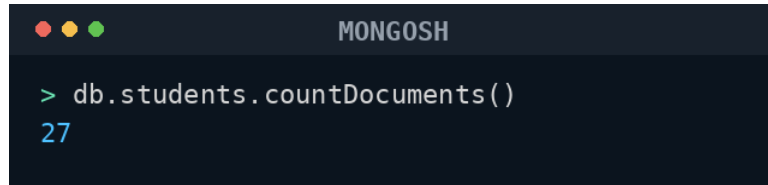
Showing 3 of 5 matching documents.

## 18. COUNT DOCUMENTS

**MongoDB Query:**

*db.students.countDocuments()*

**Explanation:** Counts documents currently present in the collection (after the delete operations).

A screenshot of a terminal window titled "MONGOSH". The terminal has a dark background with light-colored text. It shows a prompt ">" followed by the command "db.students.countDocuments()". Below the command, the output "27" is displayed in a light blue color.

```
MONGOSH
> db.students.countDocuments()
27
```

## MONGODB OPERATORS USED:

| Operator | Meaning                     | Example           |
|----------|-----------------------------|-------------------|
| \$eq     | Equal to                    | marks: {\$eq:88}  |
| \$gt     | Greater than                | marks: {\$gt:85}  |
| \$lt     | Less than                   | marks: {\$lt:60}  |
| \$gte    | Greater than or equal to    | marks: {\$gte:90} |
| \$lte    | Less than or equal to       | marks: {\$lte:70} |
| \$and    | All conditions true         | \$and:[...]       |
| \$or     | At least one condition true | \$or:[...]        |

## CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Student Management System containing student information was created and different MongoDB operations were performed on the data.

Through this project, I learned how to insert, retrieve, update, and delete documents. I also learned how to sort data in ascending and descending order, how comparison operators such as \$eq, \$gt, \$lt, \$gte, and \$lte work, along with logical operators such as \$and and \$or.

Overall, this project helped me understand how MongoDB can be used to manage and query student data efficiently.